# Scheda PASTA: app di un'azienda di sneaker

| | |
|---|---|
| **Scenario** | Un'azienda di sneaker sta costruendo un'app in cui i clienti possono registrarsi, cercare nel catalogo e pagare. Ho usato PASTA (Process for Attack Simulation and Threat Analysis) per guardare l'app dagli obiettivi di business fino ai controlli, una fase alla volta. |

| | |
|---|---|
| **I. Definire gli obiettivi di business e di sicurezza** | I clienti possono creare un profilo nell'app o accedere con un account di un altro servizio.<br>L'app accetta pagamenti, quindi gestisce dati di carte e deve rispettare PCI DSS.<br>Ricerca ed elenchi si usano di continuo, quindi il back end deve restare veloce, disponibile e corretto. |
| **II. Definire il perimetro tecnico** | Tecnologie usate dall'applicazione: interfaccia di programmazione (API), infrastruttura a chiave pubblica (PKI), SHA-256, SQL.<br>Metterei l'API al primo posto. Quasi tutto passa da lì: accesso, ricerca, pagamenti e collegamento con servizi esterni. Quindi tocca molti dati sensibili e ha la superficie d'attacco più ampia. Il database viene secondo, perché è lì che finiscono i dati, ma un'API debole apre la strada verso di esso. |
| **III. Scomporre l'applicazione** | Processo di ricerca dei prodotti, l'unico flusso che ho mappato:<br>1. Il cliente scrive una ricerca nell'app.<br>2. L'app invia la richiesta all'API.<br>3. L'API interroga il database dei prodotti.<br>4. Il database restituisce gli annunci attuali.<br>5. L'API rimanda i risultati al cliente.<br>Un vero diagramma dei flussi di dati per un'app così è molto più grande. Questo è un solo processo. |
| **IV. Analisi delle minacce** | Esterna: SQL injection tramite il campo di ricerca o un altro input, per leggere o modificare i dati.<br>Interna: un dipendente o collaboratore con più accessi di quelli che servono al suo lavoro, che copia o modifica i dati dei clienti di proposito o per errore. |
| **V. Analisi delle vulnerabilità** | Query al database costruite incollando l'input dell'utente nel testo SQL, invece di usare istruzioni preparate (prepared statements).<br>Token API che non scadono mai e non sono legati a una sola sessione, quindi un token rubato continua a funzionare finché l'attaccante vuole. |
| **VI. Modellazione degli attacchi** | Obiettivo: rubare i dati degli utenti.<br>Ramo 1: SQL injection, possibile perché le query non usano prepared statements.<br>Ramo 2: furto di un account con un token di sessione rubato o una password debole, possibile perché i token non scadono e le password non vengono controllate. |
| **VII. Analisi del rischio e** | 1. Prepared statements e validazione dell'input, per chiudere la strada all'injection. |

| | |
|---|---|
| **dell'impatto** | 2. Principio del privilegio minimo, così un account raggiunge solo i dati che gli servono.<br>3. Una policy delle password con autenticazione a più fattori, e token che scadono.<br>4. Registrazione degli eventi e procedura di risposta agli incidenti, per accorgersi in fretta di un attacco e gestirlo. |

## Cosa ne ho ricavato

PASTA parte da ciò che serve al business e non dalla tecnologia, e questo mi è piaciuto. Una volta chiari gli obiettivi è stato facile capire perché l'API meritasse più attenzione, e le due vulnerabilità scelte corrispondono ai due rami dell'albero degli attacchi. Ogni controllo finale risponde a una delle due.